

# CareerFlux

## Master Architecture & Compliance Document

*Career Intelligence, Source Discovery & Personalized Job Matching Platform*

---

Prepared by: Maruthi Sreeram Ram · B.Tech CSE, Sri Vasavi Engineering College · Class of 2027  
Document status: Design-stage / idea & architecture reference — not an active build commitment  
[github.com/maruthi-sreeram-p](https://github.com/maruthi-sreeram-p) · [linkedin.com/in/maruthi-sreeram-b57680331](https://linkedin.com/in/maruthi-sreeram-b57680331)

Version 1.0 — August 2026

# Contents

---

1. **Product Definition & Philosophy**
2. **The Problem**
3. **Candidate Onboarding**
4. **Multi-Domain Career Model**
5. **Source Intelligence — the Centerpiece**
6. **Source Access Policy Engine**
7. **Job Intelligence & Ingestion**
8. **Candidate Intelligence & Recommendation**
9. **System Architecture & Technology Stack**
10. **Data Lifecycle & Retention**
11. **Legal & Compliance Framework**
12. **Differentiation & Competitive Snapshot**
13. **Development Roadmap (Maturity Phases)**
14. **Honest Assessment & Difficulty Rating**
15. **Appendix — Registry Schema & Connector Lifecycle**

# 1 Product Definition & Philosophy

**CareerFlux** is a design-stage architecture for a career intelligence platform that automatically discovers relevant employer and job sources from a candidate's career profile, continuously ingests and normalizes job opportunities, eliminates duplicates, evaluates fit between candidates and jobs, learns individual preferences through behavior, ranks opportunities by relevance, explains why each opportunity fits, and delivers timely career alerts — while treating source access legality and candidate data protection as first-class architectural concerns rather than an afterthought.

## The core reframe

The starting insight behind CareerFlux is a shift in where the burden of the job search sits:

*“Tell me every website I should search.”*

becomes

*“Tell me what career you want. I will discover the sources.”*

This matters most outside software engineering. A developer may already know that companies publish jobs through Greenhouse, Lever, or Workday. A civil engineer, pharmacist, nurse, or financial analyst typically does not know where their industry's employers publish vacancies at all. CareerFlux's job is to carry that burden so the candidate does not have to become an expert in the employment-technology ecosystem just to find work.

## The revised philosophy

An earlier draft of this document described CareerFlux as an AI-matching product. That framing was deliberately dropped. Semantic matching and ranking are necessary, but they are commodity capability — most serious competitors in this space already have some version of them. The actual hard, valuable, defensible engineering problem is knowing *where to look* and whether it is *legitimate to look there*. The product statement is therefore:

*“CareerFlux builds and maintains an intelligent map of the employment ecosystem, then uses that map to discover and prioritize opportunities relevant to each candidate.”*

## Three intelligence layers

The system is organized around three cooperating layers, each answering a different class of question:

| Layer                  | Core Questions                                                   | Owns                                                                  |
|------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------|
| Source Intelligence    | Where to look? Can we access it? Can we trust it? Is it healthy? | Discovery, classification, access policy, registry, health monitoring |
| Job Intelligence       | What is this posting? What changed? Is it a duplicate?           | Normalization, deduplication, classification, provenance              |
| Candidate Intelligence | Who is this person? What do they want? What fits them?           | Profile extraction, preference learning, match scoring                |

All three feed a recommendation layer, which produces the candidate-facing action: an alert, a dashboard entry, or a digest item. Source Intelligence is treated as the centerpiece of the system (Section 5); Candidate Intelligence's ML components are treated as a supporting capability rather than the headline (Section 8).

## 2 The Problem

---

A candidate searching for jobs today must manually cover a fragmented set of channels: general boards (LinkedIn, Indeed), individual company career pages, ATS-hosted posting pages, startup career sites, government and industry portals, professional associations, and regional job boards. Software engineers benefit from an informal shared knowledge of which ATS platforms companies tend to use (Greenhouse, Lever, Workday, Ashby). Professionals in other fields — mechanical, civil, healthcare, finance — usually have no equivalent map at all, and are left checking company websites one at a time.

This is fundamentally a **discovery** problem before it is a **matching** problem. Most existing tools assume the candidate already knows where to search and focus their engineering effort on ranking results after the fact. CareerFlux instead treats discovery as the primary engineering challenge.

### 3 Candidate Onboarding

The default onboarding path is resume-first, to minimize friction:

```
Upload resume
  |
Resume parser
  |
Candidate profile extraction
  |
Skills / Experience / Education
Roles / Industry / Seniority / Location
  |
Structured candidate profile
  |
CareerFlux asks only for what is missing
```

A candidate may also start from a short structured form (career field, preferred roles, skills, experience range, preferred locations) if no resume is available. Optional signals — preferred companies, preferred industries, salary expectations, work mode, visa requirements, employment type — refine source prioritization but are never required to get a first result.

#### User-controlled source scope

Manual company selection is optional, layered on top of automatic discovery, not a replacement for it:

| Mode                   | Candidate says                                      | System behavior                                                         |
|------------------------|-----------------------------------------------------|-------------------------------------------------------------------------|
| Automatic              | "Java Backend Developer, India, 0–2 years"          | Sources discovered automatically from the career profile                |
| Automatic + preference | "Focus on fintech and product companies, Hyderabad" | Discovery signals are used to prioritize, not restrict, the source pool |
| Explicit watchlist     | "I specifically want Amazon, Microsoft, NVIDIA"     | Named companies are added to the automatically discovered pool          |

The candidate is never required to enumerate hundreds of companies, ATS names, or source URLs — that responsibility belongs to the system.

## 4 Multi-Domain Career Model

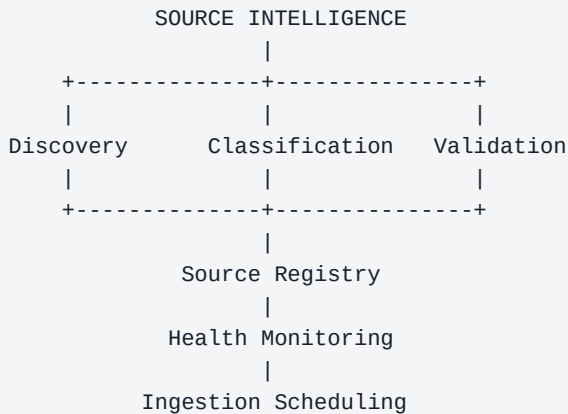
Source discovery is only meaningful if the system understands what kind of career it is discovering sources for. CareerFlux is built around an explicit career taxonomy rather than an IT-only model:

```
Career Domains
|
+-- Technology      (Software, Data, AI/ML, Cybersecurity, Cloud/DevOps)
+-- Healthcare      (Medicine, Nursing, Pharmacy, Med Lab, Admin)
+-- Finance         (Banking, Accounting, Financial Analysis, Risk, Investment)
+-- Core Engineering (Mechanical, Civil, Electrical, Electronics, Chemical, Industrial)
+-- Business        (Marketing, Sales, HR, Operations)
+-- Other Professional Domains
```

The taxonomy is a direct input to source discovery: it determines which employer categories, industry portals, and professional-association job boards are candidates for the registry for a given profile.

## 5 Source Intelligence — the Centerpiece

This is the subsystem this document treats as CareerFlux's actual differentiator. Everything else in the platform — matching, ranking, alerts — is only as good as the map of sources feeding it.



### 5.1 Discovery mechanisms

No single mechanism can find every relevant source, so discovery is layered:

**Curated registry** — an initial verified catalog of known companies and public career sources, giving the system a trustworthy starting knowledge base.

**ATS identification** — when a company is known to use a given ATS, the corresponding source adapter can be routed automatically instead of building a bespoke scraper per company.

**Career-page discovery** — for a known company, candidate paths such as /careers, /jobs, /job-openings are probed to identify the system behind the page.

**Search-assisted discovery** — search-engine queries combined with known ATS signatures surface company career pages that the curated registry does not yet cover.

**Community contribution** — external contributors can submit company, career URL, ATS, industry and adapter information; submissions are validated before being promoted into the trusted registry.

#### Deliberate limitation

CareerFlux does not claim to discover every employer on Earth from a candidate's skills. The registry is a progressively improving asset — known companies, known ATS platforms, public discovery, search discovery, and community contribution compound over time. The product statement is bounded deliberately: "CareerFlux continuously discovers and monitors an expanding ecosystem of employer and job sources relevant to your career profile," not "we monitor every company in the world."

### 5.2 Source confidence & provenance

Every ingested job carries a provenance record, independent of the access-policy state described in Section 6. This is what allows the system to answer "where did this job come from," "when did we first see it," and "has the source confirmed it is still active," which in turn is what makes deduplication and change detection trustworthy rather than opaque.

```
Job
|-- source
|-- source_url
|-- first_observed
|-- last_observed
|-- extraction_method
|-- source_confidence
|-- verification_status
`-- canonicalization_history
```

### 5.3 Continuous change detection

A source is not a one-time fetch. CareerFlux tracks state transitions on each posting over time: new job, updated job, closed job, reopened job, changed location, changed requirements. This is what allows the system to keep a candidate's view current rather than stale.

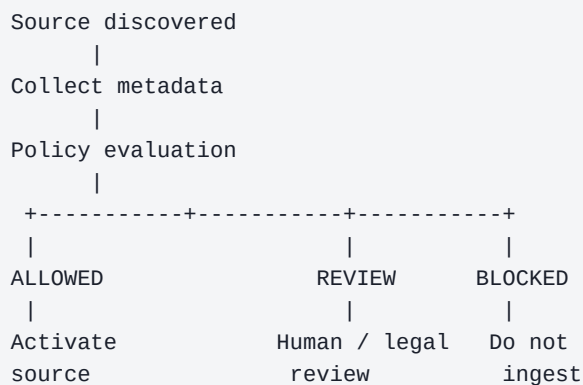
## 6 Source Access Policy Engine

This section is the most important revision in this document's history. Compliance is not expressed as a policy statement (“we comply with legal requirements”); it is expressed as an enforceable state machine that a source must pass through before it can ever be ingested.

### 6.1 The governing principle

*“A source is not considered usable merely because CareerFlux discovered it. Discovery, policy evaluation, activation, and continuous verification are separate states in the source lifecycle.”*

### 6.2 Connector lifecycle



Policy evaluation inputs include: source type, robots status, terms-of-service status, whether authentication is required, technical access restrictions (CAPTCHA, anti-bot measures), official API availability, rate limits, the type of data involved, applicable retention rules, jurisdiction, and the timestamp of the last policy verification.

### 6.3 Access decision table

| Source situation                   | CareerFlux behavior                                    |
|------------------------------------|--------------------------------------------------------|
| Official public API                | Use the API                                            |
| Public ATS endpoint                | Use the endpoint, subject to its published terms       |
| RSS / feed                         | Use the feed                                           |
| Public page, automation permitted  | Controlled retrieval, rate-limited                     |
| Login required                     | Do not automate by bypassing access                    |
| CAPTCHA / anti-bot barrier present | Do not circumvent                                      |
| Automation explicitly prohibited   | Exclude the source                                     |
| Policy unclear                     | Flag for human review — default is caution, not access |

### 6.4 Connector state machine

The registry enforces the gate structurally: no source can reach **ACTIVE** without `robots_verified = true`, `tos_reviewed = true`, and `access_policy = ALLOWED`. Anything short of that routes to **PENDING\_REVIEW** rather than defaulting to access.

```
DISCOVERED -> CLASSIFIED -> POLICY_REVIEW -> APPROVED -> ACTIVE -> HEALTH_MONITORED
```

Possible terminal / exception states:

```
PENDING_REVIEW  (metadata incomplete or policy unclear)
DEGRADED         (source health failing intermittently)
BLOCKED          (explicitly disallowed or unresolved review)
RETIRED          (source no longer relevant or permanently unavailable)
```

## 6.5 Re-verification, not permanent approval

Approval is treated as perishable. Terms of service, robots directives, and ATS API contracts change without notice; an approval granted once must not be assumed valid indefinitely.

```
Source approved
|
next_review_at reached
|
Re-check robots / ToS / API capability
|
Still valid -> remains ACTIVE
Changed/unclear -> back to REVIEW
```

The same re-verification discipline applies to ATS and API capability claims specifically — e.g. “Greenhouse exposes an unauthenticated public job-board GET endpoint” is stored as a dated, re-checkable capability record, not a permanent architectural assumption:

```
Greenhouse
|-- public_jobs_endpoint: verified
|-- auth_required: no
|-- rate_limit: last verified
|-- terms_reviewed:
`-- last_verification: 2026-08-16
```

### Honest limitation of this design

The REVIEW branch is a manual bottleneck, not an automated service — someone has to actually read a target site's terms and make a judgment call. This is an intentional constraint, not an oversight: automating away legal judgment would defeat the purpose of having the gate at all. The state machine above is a specification, not a built system; it has not been implemented as of this document's current version.

## 7 Job Intelligence & Ingestion

A source never writes directly into the primary database. Every fetch flows through an ingestion worker onto a message bus, which decouples the pace and reliability of any one source from the rest of the pipeline:

```
SOURCE -> INGESTION WORKER -> Kafka -> PROCESSING PIPELINE
```

Representative topics:

|                      |                     |                       |
|----------------------|---------------------|-----------------------|
| source.discovered    | source.sync.started | source.sync.completed |
| source.sync.failed   | job.raw             | job.normalized        |
| job.deduplicated     | job.enriched        | job.classified        |
| job.embedded         | job.matched         | job.expired           |
| notification.created | notification.sent   |                       |

This matters at scale: if several hundred sources sync around the same time, direct source-to-database writes create contention and cascading failure risk. Routing through Kafka allows normalization, deduplication, enrichment, classification, and embedding to scale independently as separate consumer groups.

### 7.1 Canonical job model

Every source represents a job differently. All ingested postings are transformed into one universal record before anything downstream touches them:

```
Job
|-- id, sourceId, companyId, externalJobId
|-- title, normalizedTitle, description
|-- location, country, employmentType
|-- experienceRange, skills, education, salary
|-- department, postedAt, updatedAt, expiresAt
|-- applicationUrl, sourceUrl, sourceType
|-- embedding
`-- status
```

### 7.2 Deduplication

The same underlying vacancy commonly appears through a company career page, its ATS, an aggregator, and search discovery simultaneously. Candidate dedup signals include company, normalized title, location, external job ID, description similarity, application URL, requisition number, and embedding similarity — collapsing multiple observations into one canonical job with a shared provenance trail (Section 5.2).

### 7.3 Extraction

Rather than treating a job description as opaque text, the system extracts role, seniority, required and preferred skills, experience range, education, location, work mode, industry, department, salary, employment type, and any visa constraints, so downstream matching operates on structured fields rather than free text.

## 8 Candidate Intelligence & Recommendation

This layer is deliberately scoped down from earlier drafts. Semantic matching, embeddings, and learning-to-rank remain necessary, but the document no longer treats them as the headline — the differentiated engineering effort lives in Sections 5 and 6, not here.

### 8.1 What this layer does

- Represents the candidate profile and each canonical job in a comparable structured form.
- Produces a compatibility signal across role, skill, experience, location, and seniority fit.
- Learns from behavior over time — viewed, saved, ignored, applied, rejected, dismissed — to adjust ranking toward what the candidate actually engages with.
- Explains the result rather than presenting an opaque score: which requirements matched, and which are a likely gap.

Example of the explanation format that keeps a match score useful rather than mysterious:

Why this matches you

Strong Java requirement match  
Spring Boot experience matches  
Experience range matches  
Location matches  
Backend role matches

Potential gap:

AWS production experience

### 8.2 Notification thresholds

Not every match is worth interrupting the candidate for. A scoring threshold determines urgency:

| Match score | Behavior                              |
|-------------|---------------------------------------|
| 95 – 100%   | Immediate alert                       |
| 85 – 94%    | High-priority notification            |
| 70 – 84%    | Included in daily digest              |
| Below 70%   | Generally hidden from active surfaces |



## Technology stack

| Layer            | Technology                                                       |
|------------------|------------------------------------------------------------------|
| Frontend         | React, Vite, Tailwind CSS, shadcn/ui                             |
| Core backend     | Java, Spring Boot                                                |
| Security         | Spring Security, OAuth2 / OIDC                                   |
| Primary database | PostgreSQL                                                       |
| Vector search    | pgvector (deferred dedicated vector DB until justified by scale) |
| Cache            | Redis                                                            |
| Messaging        | Apache Kafka                                                     |
| Search           | OpenSearch                                                       |
| ML service       | Python, FastAPI, scikit-learn, PyTorch, sentence-transformers    |

| Layer                      | Technology                                                   |
|----------------------------|--------------------------------------------------------------|
| LLM integration            | Provider-abstraction layer (not tied to one vendor)          |
| Scheduling                 | Quartz / Spring Scheduler                                    |
| Containers / orchestration | Docker; Kubernetes (EKS) at production maturity              |
| IaC / CI-CD                | Terraform; GitHub Actions                                    |
| Observability              | Prometheus, Grafana, OpenTelemetry                           |
| Testing                    | JUnit, Mockito, Testcontainers, REST Assured, Playwright, k6 |

## Why Spring Boot carries the core

Spring Boot owns authentication, user management, profiles, companies, sources, job APIs, preferences, notifications, business logic, Kafka producers/consumers, scheduling, and administration — matching the intended backend specialization. Python is introduced only where its ML ecosystem provides a clear, specific advantage (embeddings, classification models). AI is deliberately not used for deterministic work: authentication, scheduling, source health, deduplication logic, rate limiting, state transitions, and access control remain ordinary code.

## 10 Data Lifecycle & Retention

Three categories of data are tracked separately because they carry different rights, sensitivity, and retention obligations:

- **Candidate data** — sensitive, user-controlled (resume, profile, preferences).
- **Employer / job metadata** — different retention and rights considerations; largely factual.
- **Model-generated derived data** — embeddings, inferred profiles, match features. Still traceable back to an individual, and therefore explicitly included in deletion scope rather than left behind when a candidate deletes their account.

### Candidate data lifecycle

```
Upload -> Parse -> Structured profile -> Encrypted storage
  -> Used for recommendations
  -> User requests deletion
  -> Delete raw resume
  -> Delete derived personal data (profile, embeddings, match features)
  -> Delete backups per retention window
```

### Job data lifecycle

```
Active job -> Expired / closed -> Historical retention window
  -> Archived as aggregate statistics -> Purged where required
```

## 11 Legal & Compliance Framework

### Not legal advice

This section is a practical map for design purposes, based on general, publicly available information as of mid-2026. It is not a substitute for review by qualified counsel, and the law in this area — especially around AI, scraping, and India's DPDP Act — is actively developing. Counsel review is a prerequisite before any real public launch, not an optional step.

### 11.1 Copyright

Facts and data — job title, company, location, salary figures, listed requirements — are not copyrightable. The specific creative expression of a job description (the written paragraphs themselves) generally is. This has a direct architectural consequence: extracting structured facts into the canonical job model (Section 7.1) is low-risk. Storing and redisplaying full verbatim description text is higher-risk and should default to summarization or linking back to the source rather than wholesale republication.

### 11.2 Unauthorized access law (CFAA-style statutes)

In the U.S., the governing question under the Computer Fraud and Abuse Act is what “without authorization” means. Courts, notably in *hiQ Labs v. LinkedIn*, have held that accessing publicly visible, logged-out data does not itself violate anti-hacking statutes. The practical implication: staying logged out and never bypassing a technical access control (login walls, CAPTCHAs) keeps a source on solid ground for this specific class of risk. This is exactly the logic encoded in the access decision table in Section 6.3.

### 11.3 Terms of Service (contract law)

A Terms-of-Service violation is a separate legal question from unauthorized access, and does not automatically become a criminal matter. It can, however, support a breach-of-contract claim, particularly where the terms were affirmatively agreed to (clickwrap) rather than merely posted (browsewrap). This is why the Section 6 policy engine treats `tos_status` as a required, independently-tracked field rather than folding it into a general “access allowed” flag, and why the priority order in Section 6.3 favors official APIs and public endpoints, which sidestep this risk category almost entirely.

### 11.4 Personal data / privacy law

Any data point that identifies or could reasonably identify a person — a name, an email, a job title paired with a specific company — is personal data under GDPR-style frameworks regardless of how publicly it was posted. This is directly relevant to CareerFlux because candidate resumes and profiles are personal data by definition.

For an India-based product, the applicable framework is the Digital Personal Data Protection (DPDP) Act, enacted 2023, with enforcement maturing through 2025–2026. Its GDPR-style obligations — lawful basis for processing, purpose limitation, and defined retention/deletion — apply to CareerFlux directly, independent of where any target employer or ATS is based.

### 11.5 Compliance mechanisms baked into the architecture

Rather than betting the platform's architecture on today's specific regulatory interpretation, the design keeps the underlying mechanisms general, so that a future reinterpretation becomes a policy or configuration change rather than a redesign:

- Consent
- Purpose limitation

- Retention windows
- Deletion (including derived data — Section 10)
- Access controls
- Audit logging
- Data minimization

## 11.6 Working compliance posture

| Practice                                      | Why                                                                  |
|-----------------------------------------------|----------------------------------------------------------------------|
| Stay logged out where possible                | Keeps access on solid ground under CFAA-style analysis               |
| Prefer official APIs / public endpoints first | Sidesteps ToS and access-control risk almost entirely                |
| Honor robots.txt and rate limits              | Baseline good-faith signal referenced across most current case law   |
| Never bypass logins, CAPTCHAs, or paywalls    | The clearest line between defensible and indefensible access         |
| Minimize personal data collected and retained | Reduces exposure under DPDP / GDPR-style frameworks                  |
| Separate collection from redistribution       | Storing facts is different from republishing creative text wholesale |
| Document every access decision, dated         | Makes the compliance posture auditable, not just asserted            |

# 12      Differentiation & Competitive Snapshot

**Snapshot, not permanent claim**

This comparison reflects the competitive landscape as understood at the time of writing (August 2026) and should be re-evaluated periodically rather than treated as a fixed claim of differentiation. “AI job aggregator” alone is not a sufficient differentiator in this market.

Two products are the closest reference points. JobRadar uses search operators and known ATS signatures to find postings directly on company career pages and ATS platforms. JobSentinel is the closer technical comparison — its current feature set includes source health checks, source classification, deduplication, resume matching, embeddings/reranking, alerts, and application-related functionality, and its direction overlaps significantly with CareerFlux’s own.

**Where CareerFlux differentiates**

| Differentiator                      | What it means concretely                                                                                     |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Automatic career-source discovery   | The candidate specifies intent; the system discovers relevant sources rather than requiring a pre-built list |
| Multi-domain career intelligence    | Explicit taxonomy across Technology, Healthcare, Finance, Core Engineering, Business — not IT-only           |
| Source Access Policy Engine         | Legal/compliance posture enforced as connector state, not documentation                                      |
| Source confidence & provenance      | Every job traceable to origin, first/last observed, extraction method                                        |
| Continuous change detection         | New / updated / closed / reopened status tracked per posting, not a one-time snapshot                        |
| Explainable matching                | Match scores accompanied by concrete matched requirements and gaps                                           |
| Production ingestion infrastructure | Kafka-based pipeline with retry, dead-letter handling, idempotency, per-source health                        |

## 13 Development Roadmap (Maturity Phases)

Deliberately expressed as maturity phases rather than a fixed schedule — this document describes the shape of the system, not a delivery commitment.

| Phase                | Delivers                                                                                                                                                                                                                                                          |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Foundation           | Repository, architecture docs, Docker Compose, database schema, CI pipeline, coding standards                                                                                                                                                                     |
| Candidate Platform   | Authentication, resume upload, profile extraction, career taxonomy, preferences                                                                                                                                                                                   |
| Source Intelligence  | Discover source -> Identify source type -> Determine access mechanism -> Verify robots/access policy -> Review applicable terms -> Define allowed data fields -> Define retention policy -> Record verification timestamp -> Approve source -> Activate connector |
| Ingestion Framework  | Source adapters, scheduler, Kafka topics, workers, retry, idempotency                                                                                                                                                                                             |
| Job Intelligence     | Normalization, deduplication, skill extraction, classification, search indexing                                                                                                                                                                                   |
| Recommendation       | Embeddings, feature generation, match scoring, explainability, ranking, behavioral feedback                                                                                                                                                                       |
| Alerts               | Email, push/web notifications, digest, priority alerts                                                                                                                                                                                                            |
| Production Hardening | Security review, observability, load testing, failure recovery, cost controls                                                                                                                                                                                     |
| Scale                | Cloud deployment, Kubernetes, Terraform, CI/CD, monitoring at growing source counts                                                                                                                                                                               |

### Hard gate

Source Intelligence (Phase 3 above) is placed explicitly before Ingestion in dependency order — no connector reaches the ingestion framework without first clearing the Section 6 policy gate. This is a design constraint, not a suggestion.

### Source scale progression

| Stage   | Trusted sources |
|---------|-----------------|
| Phase 1 | 20 – 30         |
| Phase 2 | 50 – 100        |
| Phase 3 | 200+            |
| Phase 4 | 500+            |
| Phase 5 | 1000+           |

The architecture is intended to support this growth without requiring a rewrite between stages — the adapter interface (Section 7) and the registry schema (Appendix) are the abstractions that make that possible.

## 14 Honest Assessment & Difficulty Rating

This section exists because a design document that only lists strengths is not useful to revisit later. The following is a candid appraisal, not a pitch.

### Where the design is genuinely strong

- The reframe from “matching” to “discovery + legitimate access” as the core hard problem is correct and not the obvious framing most competitors reach for.
- The Source Access Policy Engine turns compliance from a paragraph of prose into an enforceable state machine — a materially better design than the original draft of this document.
- The adapter interface keeps the core pipeline decoupled from any individual source's quirks, which is the right abstraction for the planned scale progression.
- Provenance and re-verification (Sections 5.2 and 6.5) show an understanding that trust in a data source decays over time and must be actively maintained, not assumed.

### Where this document should be read carefully

- Nothing described here is built. This is a specification, and the gap between a well-specified policy engine and a working one is substantial — expect real edge cases (in-flight syncs when a source flips to REVIEW mid-run, who performs the human review step and on what timeline) to surface only once implementation starts.
- The REVIEW branch in the connector lifecycle is a manual bottleneck by design. It does not scale the way the rest of the pipeline does, and that constraint should be stated plainly whenever this document is discussed, rather than glossed over.
- Estimated engineering difficulty across the full vision remains high: source discovery and reliable heterogeneous ingestion are the hardest components, ahead of matching, ranking, or production reliability work individually.
- As a build commitment for one developer on a tight timeline, this is over-scoped. As a design artifact — something to revisit, mine for a scoped MVP, or discuss in an interview — it is sound and worth keeping.

### Recommended difficulty ranking of subsystems

| Subsystem                        | Relative difficulty                                                          |
|----------------------------------|------------------------------------------------------------------------------|
| Automatic source discovery       | Highest — the central unsolved problem this document is built around         |
| Production reliability at scale  | High — failures, retries, and partial outages compound quickly               |
| Reliable heterogeneous ingestion | High — every source exposes a different structure and access model           |
| Personalized ranking             | Moderate-high — requires behavioral data the system does not have on day one |
| Deduplication                    | Moderate — tractable with good provenance data, still non-trivial at scale   |
| Job understanding / extraction   | Moderate — consistent extraction from free-text descriptions                 |

## 15 Appendix — Registry Schema & Connector Lifecycle

### Source registry — enforceable schema

```
Source
|-- id
|-- company
|-- source_url
|-- source_type
|-- ats_provider
|-- discovery_method
|
|-- robots_status
|-- robots_verified_at
|-- tos_status
|-- tos_reviewed_at
|-- access_policy_status
|-- access_reviewed_at
|
|-- rate_limit_policy
|-- allowed_data_fields
|-- retention_policy
|
|-- status                (DISCOVERED / CLASSIFIED / POLICY_REVIEW /
                           APPROVED / ACTIVE / DEGRADED / BLOCKED / RETIRED)
|-- health_status
|-- last_verified_at
`-- next_review_at
```

### Enforcement rule

```
DISCOVERED
|
required metadata collected
|
policy checks complete
|
robots_verified = true
tos_reviewed = true
access_policy = ALLOWED
|
v
ACTIVE

Any unmet condition -> PENDING_REVIEW (never defaults to ACTIVE)
```

### Source adapter interface

```
public interface JobSourceAdapter {  
  
    SourceMetadata getMetadata();  
  
    List fetchJobs(SourceConfiguration config);  
  
    SourceHealth checkHealth(SourceConfiguration config);  
}
```

Representative adapters:

|                        |                        |                           |
|------------------------|------------------------|---------------------------|
| GreenhouseAdapter      | LeverAdapter           | AshbyAdapter              |
| SmartRecruitersAdapter | WorkableAdapter        | OfficialApiAdapter        |
| FeedAdapter            | SearchDiscoveryAdapter | CustomPublicSourceAdapter |

---

*End of document. This is a living design reference — intended to be revisited, corrected, and re-scoped rather than treated as final.*